

Module 5: Code Optimization and Code Generation

Question 9 (3 marks each):

- **A-9:** Explain code motion with an example.
- **B-9:** Construct the syntax tree and draw the DAG for the statement:
$$e := (a * b) + (c - d) * (a * b).$$
- **D-9:** Differentiate local and global optimizations.
- **E-9:** Construct the syntax tree and draw the DAG for the statement:
$$d + a * (b - c) + (b - c) * d.$$
- **F-9:** Differentiate local and global optimization in compiler design.
- **July 2023-9:** Explain common subexpression elimination with an example.

Question 10 (3 marks each):

- **A-10:** Write the algorithm for partitioning a sequence of three-address instructions into basic blocks.
- **B-10:** Explain any three issues in the design of a code generator.
- **D-10:** Illustrate the role of register descriptors and address descriptors in the code generation phase.
- **E-10:** With a suitable example, explain the induction variable elimination technique for loop optimization.
- **F-10:** Write the algorithm for partitioning a sequence of three-address instructions into basic blocks.
- **July 2023-10:** How does the peephole optimization technique contribute to the compilation process?

Question 19 (14 marks each):

- **A-19:**
 - a) With suitable examples, explain loop optimization techniques.
 - b) With a suitable example of a basic block, explain the code-improving transformations of a basic block.
- **B-19:**
 - a) With suitable examples, explain the following loop optimization techniques: (i) Code motion, (ii) Induction variable elimination, (iii) Strength reduction.
 - b) Explain the optimization of basic blocks.
- ****偶, D-19:**
 - a) Explain any three code optimization transformations.
 - b) Perform common subexpression elimination for the three-address code:

```
text Copy
t1 = a + b
x = t1
t2 = a + b
t3 = t2 + c
b = t2
t4 = a + b
y = t4
```

and represent it as a quadruple table.

- **E-19:**
 - a) Explain any four principal sources of optimization.
 - b) What is a basic block? Explain the structure-preserving transformations on a basic block.
- **F-19:**
 - a) With suitable examples, explain the following loop optimization techniques: (i) Code motion, (ii) Induction variable elimination, (iii) Strength reduction.
 - b) What is peephole optimization? Explain any three transformations done in peephole optimization.
- **July 2023-19:**
 - a) Explain different code optimization techniques.
 - b) Generate a code sequence for the assignment $d = (a - b) + (a - c) + (a - c).$

Question 20 (14 marks each):

- **A-20:**
 - a) Explain issues in the design of a code generator.
 - b) Write the code generation algorithm. Using this algorithm, generate a code sequence for the expression $x = (a - b) + (a + c) + (a + c).$
- **B-20:**
 - a) For the C statement $S := A - B + C * D - E + F$, write the three-address code and quadruples. Also, convert the three-address code into machine code.
 - b) Write the code generation algorithm and explain the `getreg` function.
- **D-20:**
 - a) Neatly explain the code generation algorithm and `getreg` function.
 - b) Convert to three-address code and write machine code for the statement:
$$x = a / b + a / b * (c - d).$$
- **E-20:**
 - a) Explain peephole optimization techniques with an example.
 - b) Explain any three issues in the design of a code generator.
- **F-20:**
 - a) Explain any three issues in the design of a code generator.
 - b) Translate the expression $W := (A - B) + (A - C) + (A - C)$ into three-address code and generate the machine code for the three-address code.
- **July 2023-20:**
 - a) Explain the design issues of a code generator.
 - b) Illustrate the optimization of basic blocks with examples.

This organization groups all questions from the six question papers by module, making it easier to study or analyze the content module-wise. Each question is clearly labeled with its source paper and question number for traceability. If you need further assistance, such as detailed solutions to specific questions or additional analysis, please let me know!



How can Grok help?

  DeepSearch  Think

Grok 3 